

Sortowanie przez wybór

„[] opanowanie teorii jest zaledwie niezbędnym wstępem”

Opis algorytmu

- Algorytm "wybiera" najmniejszy element i "odkłada" go na początek. Przypomina to układanie kart – za każdym razem wyszukuje się najmniejszą kartę, aby odłożyć ją na lewo

Metoda

Metoda

- Logiczny podział tablicy na dwie części:
 - część posortowaną – na początku pusta
 - część nieposortowana – na początku cała tablica

Metoda

- Dla pozycji i od 0 do $n-1$:
 - zakłada się, że najmniejszy element znajduje się na pozycji i (na początku na pozycji 0)

Pierwszy wyraz nieposortowanego ciągu



$T[5]=\{8,2,0,1,6\}$



Zakładamy, że to jest minimum

Metoda

- Dla pozycji i od 0 do $n-1$:
 - przeszukuje się pozostałą część tablicy ($n + 1 \dots n-1$), aby znaleźć indeks najmniejszego elementu – jeśli taki jest

Pierwszy wyraz nieposortowanego ciągu



$T[5]=\{8,2,0,1,6\}$



Minimum – indeks 2

Metoda

- Dla pozycji i od 0 do $n-1$:
 - zamienia się element z pozycji i z tym najmniejszym

$T[5] = \{8, 2, 0, 1, 6\}$



Ciąg posortowany



$T[5] = \{0, 2, 8, 1, 6\}$



Ciąg nieposortowany

Algorytm

Algorytm

```
dla i ← 0, 1, ..., n - 2 wykonuj
    m ← i
    dla j ← i + 1, i + 2, ..., n - 1 wykonuj
        jeśli A[j] < A[m] to m ← j
    pom ← A[i]
    A[i] ← A[m]
    A[m] ← pom
```

Specyfikacja

Specyfikacja

Dane:

- Liczba naturalna: $n > 0$ (liczba elementów tablicy T)

Wynik:

- Posortowana niemalejąco n -elementowa tablica jednowymiarowa zawierająca liczby naturalne: $T[0...n-1]$

Implementacja

```
void selectionSort(int array[], int n)
{
    int m, temp;

    for (int i = 0; i <= n - 2; i++)
    {
        m = i;

        for (int j = i + 1; j <= n - 1; j++)
        {
            if (array[j] < array[m])
                m = j;
        }

        temp = array[i];
        array[i] = array[m];
        array[m] = temp;
    }
}
```

```
int main()
{
    int array[] = { 29, 10, 14, 37, 13, 2, 9, 15, 28, 28 };
    int n = sizeof(array) / sizeof(array[0]);

    selectionSort(array, n);

    std::cout << "Sorted array: \n";
    for (int i = 0; i < n; ++i) {
        std::cout << array[i] << " ";
    }

    std::cout << std::endl;
    return 0;
}
```

Złożoność

Złożoność czasowa

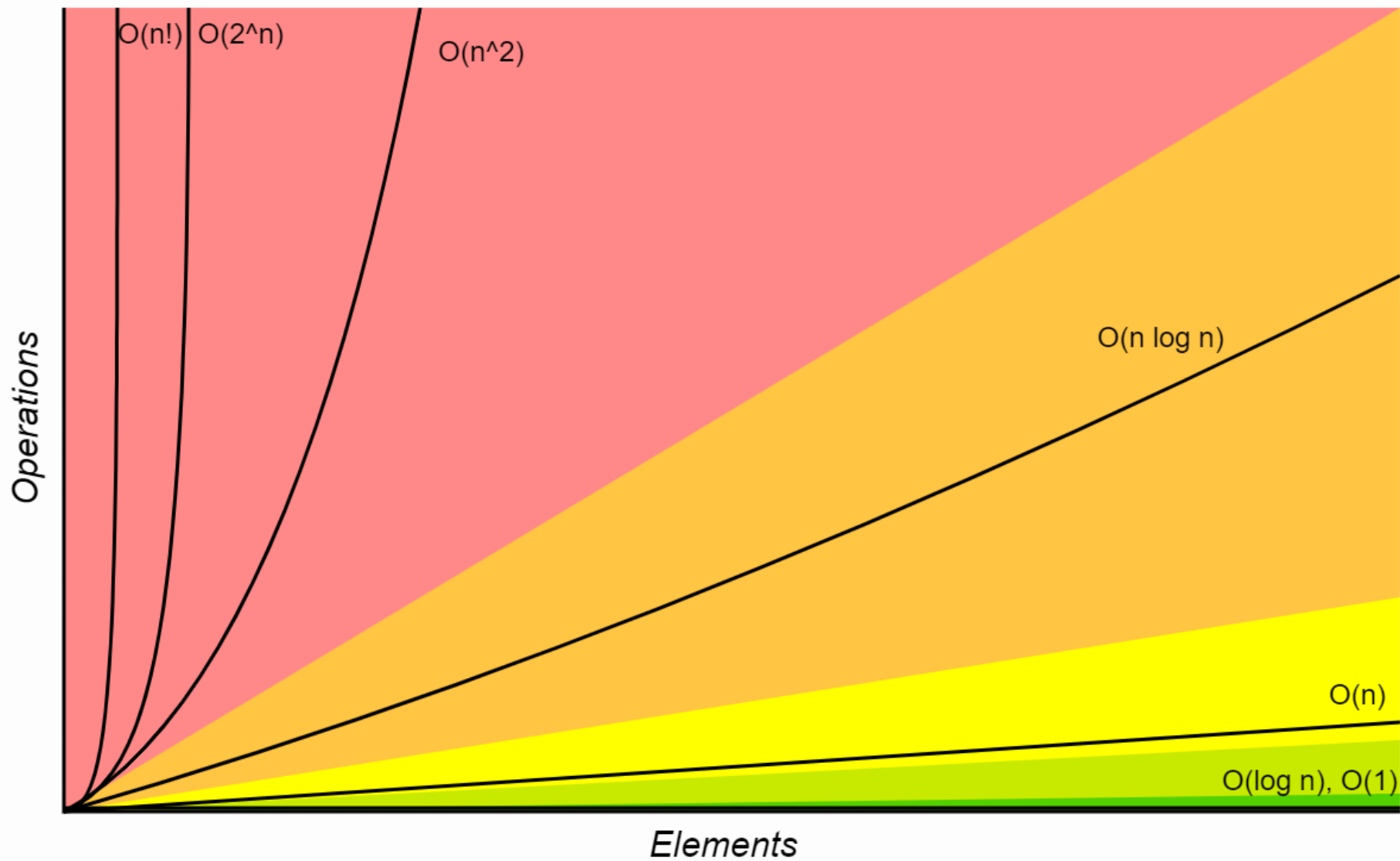
- Złożoność metody sortowania przez wybór określamy przez wyznaczenie liczby wystąpień operacji dominującej, którą jest **porównanie**. W n-wyrazowym ciągu w kolejnych n-1 fazach algorytmu szukamy minimum w coraz krótszym ciągu, stąd liczba porównań wynosi:

$$(n-1) + (n-2) + \dots + 1 = \frac{n(n-1)}{2}$$

Złożoność czasowa

- średnia: $O(n^2)$
- najgorsza: $O(n^2)$
- najlepsza: $O(n^2)$

Horrible Bad Fair Good Excellent



www.szymkowiak.tychy.pl

KONIEC

Grzegorz Szymkowiak